



CAB420 ASSESSMENT 1A REPORT

RORY BLAIR, N10773398



MARCH 31, 2025

CAB420
MACHINE LEARNING

PROBLEM 1:

Pre-Processing:

Processing data is an essential part of preparing a model for training, especially models used for predicting an outcome. The data was collected from their respective '.csv' files, separated into their respective sets and then standardized. The standardization of the data sets was completed using a function provided in the week 2 practical [1]. Although standardization is not required for linear regression, it is necessary for the ridge and lasso models as both models are sensitive to the scale of data. Standardizing the data prevents bias towards certain variables. [2] No other processing such as the removal of outliers was not undertaken as doing so could result in bias. Therefore, the standardised data is used for all 3 models to make sure models are tested on the same data.

Selection of Lambda values:

The selection of the lambda values is crucial in the ridge and lasso models as the lambda value is used to ensure a balance between overfitting and underfitting by managing variance and bias in the model's coefficient. A range of values from 10^{-10} to 1 was selected as appropriate values for lambda, which is a common range, and is considered a 'coarse search'. This is the same range of lambda values given except the number of values created has been increased from the default, 50, to 100 to allow more values to be tested. This range of lambda values allows for experimentation of strong and weak regularisation, thus allowing a balance between overfitting and underfitting. If done correctly, a value of lambda would be found that does not lose its prediction accuracy by lowering the Root Mean Square Error (RMSE) on the validation set.

Compare 3 Models:

All three models were trained and evaluated using the same standardized data. The linear regression model was adapted from the week 1 practical [3] whilst the ridge and lasso models were borrowed from the week 2 practical, specifically Questions 2 and 3. [1] The evaluation methods were also borrowed from these weeks allowing the exploration of a range of different lambda values and finding the best lambda values and their corresponding measures of fit. A comparison of the 3 models can be seen below in Table 1.1. This table shows the lambda values, RMSE, R^2 and Adjusted R^2 (See Appendix 1D, 1E & 1F for model summary table)

	Linear Regression	Ridge	Lasso
Lambda	NONE	0.061	0.002
RMSE	0.730	0.142	0.144
R²	0.759	0.718	0.706
Adjusted R²	0.637	0.575	0.557

Table 1.1: Problem 1 Model Comparison

As shown in Table 1.1 above, the linear regression model has the highest values in all three measures of fit, while the ridge and lasso models appear to be very similar. As noted the R^2 and R^2 Adjusted values are high indicating that the model is good at describing the data variance. The only problem with the linear regression model is that it has a very high RMSE value suggesting the model would make mistakes, thus overfitting to the training data and affecting its prediction accuracy [4].

Although the ridge and lasso models do not have higher R^2 and R^2 Adjusted values their values are still considered reasonable. Unlike the linear regression model and its high RMSE values, both the ridge and lasso models have considerably low RMSE values. Low RMSE values indicate that the model makes more accurate predictions [4]. The differences in prediction accuracy can be visually represented as residual plots as seen in Figures 1.1, 1.2, and 1.3 below. (See Appendix 1A for visual similarities in R^2 values via Q-Q plots) Between Ridge and Lasso, the models seemed to be partially accurate when it comes to the Test Set performance (See Appendix 1B for the test performance for Ridge and Appendix 1C for the test performance for Lasso)

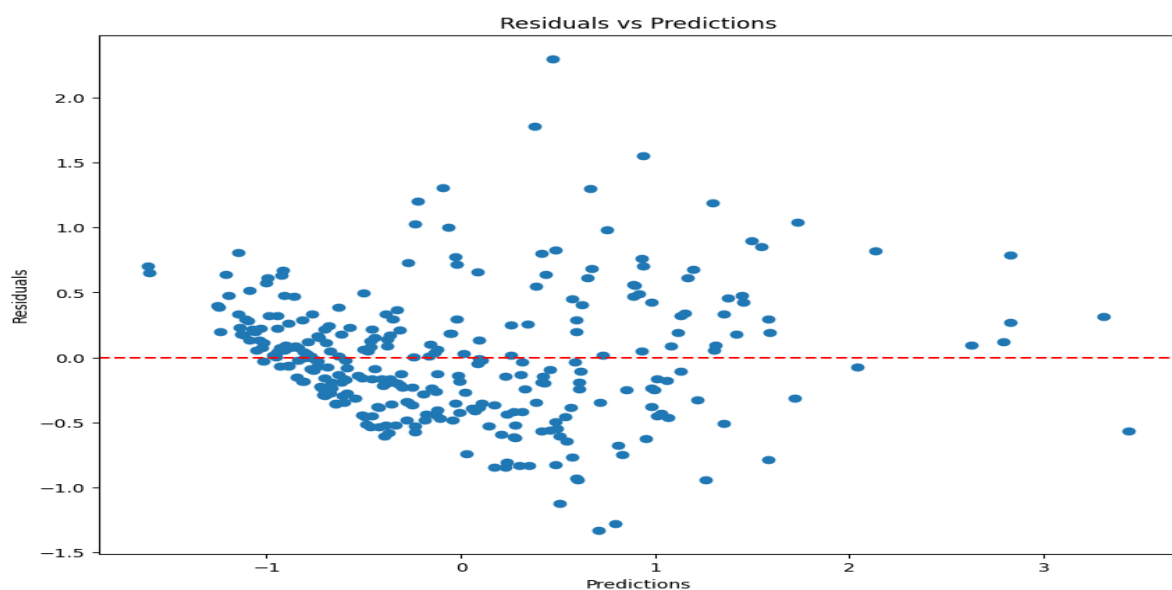


Figure 1.1: Plot of Residuals for Linear model

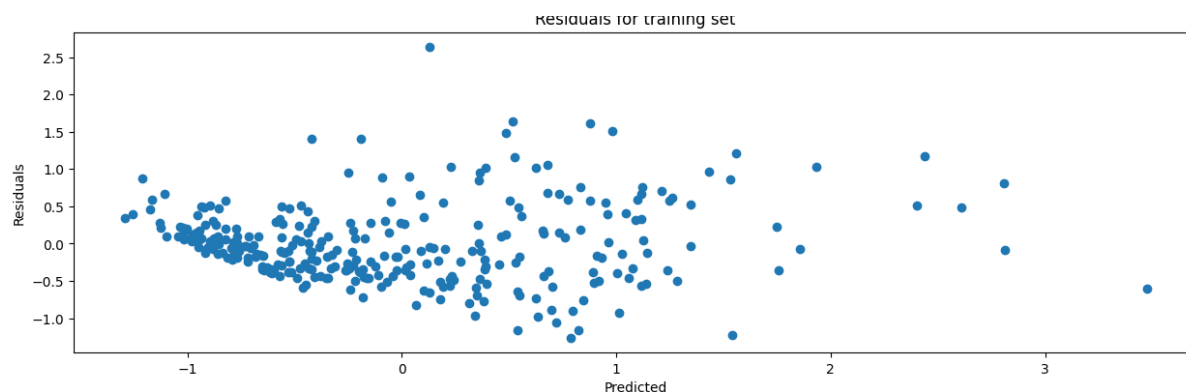


Figure 1.2: Plot of Residuals for Ridge model

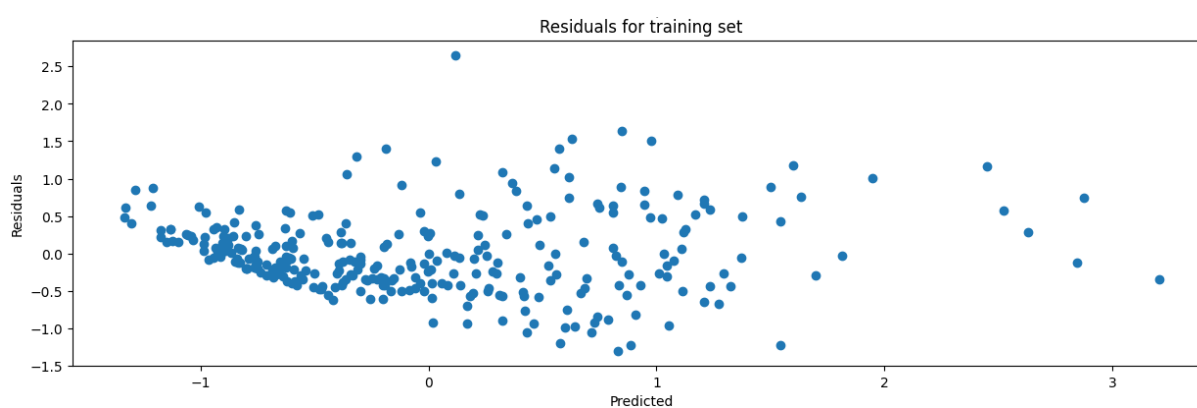


Figure 1.3: Plot of Residuals for Lasso model

One testing assumption that can be made is about the residual distribution, which was tested using both Q-Q plots and more precisely residuals for training sets. As seen in Figure 1.1, 1.2 and 1.3 the plots reveal no clear patterns meaning that the residuals were reasonably distributed. Although the residuals were reasonably distributed the RMSE value for the linear model was high, suggesting that it did not work well. (See appendix 4A for code to replicate similar results for Problem 1)

Ethical Concerns:

When creating prediction models, topics such as crime, demographics, and socio-economic need ethical considerations when determining the results of a prediction model. One of the most important considerations is the fairness/bias of the results. For example, the data could have biases toward certain communities, thus the models could make biased/unfair predictions. The final consideration is the model accountability and making sure the model is validated and monitored to prevent biased data from leading to serious consequences, such as misallocation of resources and false accusations. [5]

PROBLEM 2:

Pre-Processing:

Unlike the first problem, no additional data processing was required as the data provided was already extracted and split into the appropriate data set. These data sets do not include missing values, are all numerical, and are already useable. No scaling or normalisation was performed to improve performance. Scaling is not required as the range of data is satisfactory. Normalisation was not necessary as undesirable data, such as outliers, should be included to provide information about how well the model distinguishes the classes. Removing the undesirable data could lead to crucial information being lost as the model will not include the few outliers to really test the models.

Hyper-parameter selection:

All three models used a 'gridSearch' method and a 'Trial-and-Error' (T&E) method to find hyper-parameters. The 'gridSearch' method is used to find optimal hyper-parameters based on the parameters that are tested within the 'gridSearch'. This method was borrowed from the week 3 practical [6]. The T&E method worked by starting off with testing one hyper-parameter, finding the best option and then testing with that optimal hyper-parameter and a different hyper-parameter. This method was repeated until an optimal value was found for all hyper-parameters.

For the K-Nearest Neighbour (KNN), there were three hyper-parameters that were tested, as shown in Table 2.1 below. Usually, a small value for 'n_neighbour' can lead to overfitting, whilst larger values could lead to underfitting [7]. This is the reason the list of neighbours starts at 3 as if it started at 1 or 2, they were chosen, thus overfitting the model.

Hyper-Parameter	Options
n_neighbours	list(range(3,198))
Weights	'uniform' 'distance'
Metric	'minkowski' 'manhattan'

Table 2.1: KNN Hyper-Parameter Option

For the Support Vector Machine (SVM) model there were four hyper-parameters that were tested, which are shown in Table 2.2 below. Usually a smaller ‘C value’ puts more attention on maximising the margin instead of classifying the training correctly, meaning there are more errors in classification. On the other hand, if the ‘C value’ is large the opposite happens. However, this can lead to overfitting, which is the reason the tested ‘C value’ ranges from 0.01 to 1000.

Hyper-Parameter	Options
C	[0.01, 0.1, 1, 10, 100, 1000]
Kernel	'linear' 'rbf' 'poly'
Gamma (IF Kernel = rbf)	[1, 0.1, 0.01, 0.001]
Degree (IF Kernel = Poly)	[1, 3, 4, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39]

Table 2.2: SVM Hyper-Parameter Option

For the Random Forest (RF) model there were only 2 hyper-parameters that were tested, which are shown in Table 2.3 below. The ‘n_estimator’ is the number of trees in the forest, meaning the larger the number of trees, the more accurate the model. However, the larger number usually means more computational power, thus a longer training process. The depth value is the depth that each tree will explore and works similarly to the ‘n_estimator’. The larger the depth, the more accurate the model, but a longer and slower training process [8].

Hyper-Parameter	Options
N_estimator	list(range(1,100))
Depth	list(range(10,20))

Table 2.3: RF Hyper-Parameter Option

Compare 3 Models

All three models were trained, with some models being tested twice, one ‘gridSearch’ and one T&E. The evaluation method used to compare these models was borrowed from the week 3 practical, which allows for the ‘F1’ scores and confusion matrix to be created for each model tested. [6] The comparison between all tested models is seen in Table 2.4 below, which shows the ‘F1 score’ and the corresponding confusion matrix.

	Hyper-Parameter	F1	Confusion Matrix
KNN ‘gridSearch’	'n_neighbors': 19 'weights': 'distance' 'metric': 'minkowski'	0.80	Figure 2.1
KNN T&E	'n_neighbors': 4 'weights': 'distance' 'metric': 'minkowski'	0.85	Figure 2.2
SVM ‘gridSearch’	'C': 10 'kernel': 'poly' 'degree': 15	0.81	Appendix 2A
RF ‘gridSearch’	'n_estimators': 59 'max_depth': 10	0.81	Appendix 2B
RF T&E	'n_estimators': 30 'max_depth': 5	0.83	Appendix 2C

Table 2.4: 3-Models Comparison

As seen in the table above, the models that used the T&E method seemed to have resulted in higher ‘F1’ scores than the ‘gridSearch’ method. The highest ‘F1’ score was the KNN T&E model which had a score of 0.85. the next best was the RF T&E model which had a score of 0.83. The RF and SVM models using the ‘gridSearch’ method seem to have the same ‘F1’ scores, likely occurred by chance. Finally, the least accurate model was the KNN ‘gridSearch’ model with an ‘F1’ score of 0.80. All these scores are still reasonable and there is little variance between the best and the worst models in terms of ‘F1’ scores. Going into detail about the variance between both the KNN models can be seen in Figures 2.1 and 2.2, which are the confusion matrices.

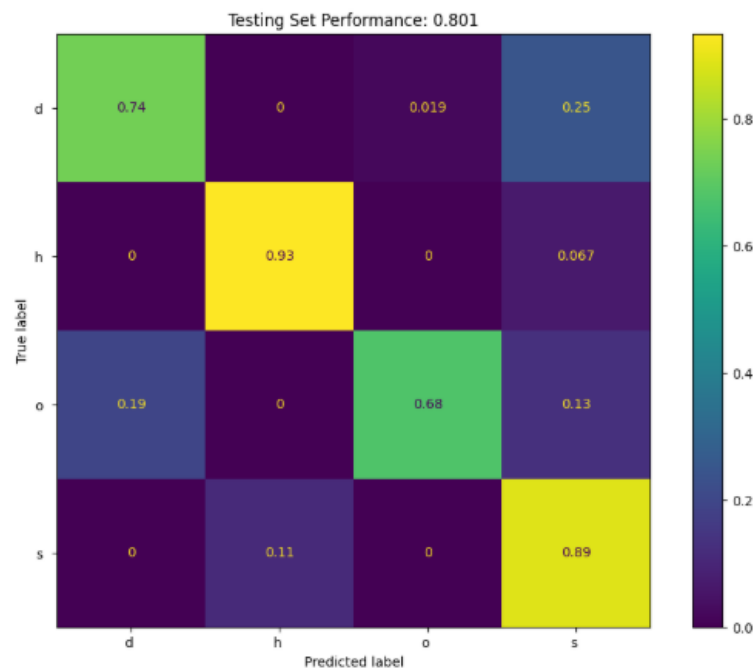


Figure 2.1: KNN (gridSearch) Confusion Matrix

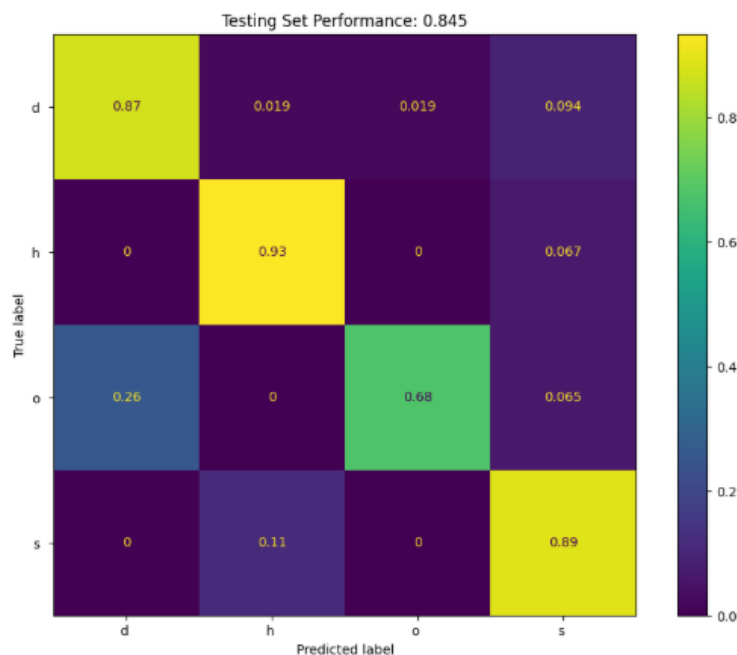


Figure 2.2: KNN (T&E) Confusion Matrix

As seen in Figures 2.1 and 2.2 above, the confusion matrices between the two different methods are very similar. As seen in the figures, the ‘S-S’, ‘O-O’, and ‘H-H’ grids are the same but change when it comes to the ‘D-D’ grid. The T&E method has a ‘D-D’ accuracy of 0.87, whilst the ‘gridSearch’ method has a ‘D-D’ accuracy of 0.74. This happens because the T&E method corrects the mistake made in the ‘gridSearch’ model, which is the ‘D-S’ grid. In the ‘gridSearch’ model the ‘D-S’ grid is 0.25, whilst in the T&E model the grid is 0.094, a huge improvement on the mistake made by the ‘gridSearch’ model. Although this problem was discovered and fixed between models, both models still have one problem with them, which is the ‘O-D’ and ‘S-H’ grids. These grids still have a major number meaning there are still problems with both models. This problem can also be discovered in the difference between the two RF models (See Appendix 2B and 2C) (See appendix 4B for code to replicate similar results for Problem 2)

PROBLEM 3:

Pre-Processing & Neural Network Design:

The only processing was to reduce the number classes in the training labels to 10 labels as the data needed to be classified into numbers ranging from 0 to 9, rather than 49.

The Deep Convolutional Neural Network (DCNN) was designed to be a simple VGG-style model. A VGG-style was chosen as it was an easy model to understand and modify. This network consists of the 3 convolutional blocks increasing from 128 to 256 to 512 with the 1st block learning patterns, the 2nd block learning shapes, and the 3rd block learning features. These were chosen as the data set given was small and these numbers allow for more efficiency and accuracy. Each convolution block is followed by batch normalisation and max pooling. Max pooling is used to help with feature extraction and reduce overfitting. After these blocks, the extracted image layers are then converted to a 1D array via flattening and then are used within the dense layer which allows for the model to learn even more features about the images. The last two layers were the dropout and ‘softmax’ where the dropout layer would help with overfitting and the ‘softmax’ would help with multiple different classifications the model would be assessed on [9] [10].

Both DCNN models are compiled using the Adam optimiser for adaptive learning rates and categorical cross-entropy loss to allow the models to display the results of each class’s probabilities [11] [12]. Both models were trained with a batch size of 32 and 100 epochs, with an ‘Early Stopping’ parameter that monitors the validation loss and has a patience value of 25. An ‘Early Stopping’ parameter allows the training to finish before 100 epochs if the validation loss starts to settle/converge, thus the accuracy of the model converges

Data Augmentations:

There were two types of data augmentations made to the data set. The first augmentation was about rotating images 30 degrees left or right. This augmentation was done by going through the multiple other augmentation options it was discovered that particular numbers within particular fonts looked different based on the angles they were rotated, particularly the numbers 1 and 7. As seen below in Figure 3.3 some rotating the number 1 clockwise could be considered a 7 and if a 7 was rotated anticlockwise it could be considered a 1. The other augmentation that was used was the inverted colour, as maybe when colour is inverted, the colours of a particular photo may blend together, thus making it hard for the model.



Figure 3.1: Rotation Augmentation Example (Left = 7, Right = 1)

Compare 3 Models:

The SVM and the 2 DCNN models were trained and tested. The evaluation method used to assess the models was taken from the week 4 & 5 practical. This evaluation method allowed for the separate epochs and their loss and accuracy values to be printed out. This method also provided the Training vs Validation plots to be created for both accuracy and loss. The final information that this method also displays is the confusion matrix to allow for the analysis of the overall model. The comparison between the three models can be seen in Table 3.1 below, which shows the number of epochs, the 'F1' scores, training time, testing time, and their corresponding confusion matrices.

Models	Epochs	F1	Trian Time	Test Time	Confusion Matrix
SVM	NONE	0.19	4.37	11.17	Train = Appendix 3A Testing = Figure 3.2
DCNN	68/100	0.83	52.54	6.53	Train = Appendix 3B Testing = Figure 3.3
Augmented DCNN	100/100	0.55	58.72	5.79	Train = Appendix 3C Testing = Figure 3.4

*Table 3.1: 3-Model Comparison
(Google Colab T4 GPU with available compute units)*

As seen in Table 3.1 above, the best model was the DCNN model with the unaugmented training data, with the next best being the DCNN with augmented data and the SVM model being the worst. Although the unaugmented DCNN had the highest training time, it got the most accurate result with a 0.83 and without overfitting the model entirely [13]. The unaugmented DCNN model could be modified to allow for a quicker training time but lose some accuracy. The augmented DCNN has an 'F1' score of 0.55, which is a reasonable score considering the augmented data that it was trained on. The SVM model was considered a failure as the model was chaotic in classifying the true positives.

As seen in Figures 3.2, 3.3 and 3.4, the unaugmented DCNN has a near-perfect diagonal line from top left to bottom right, with some double digit numbers in the other corners. The augmented DCNN is only able to correctly classify 1, 2, 4 and 5 properly and then has trouble classifying the rest of the numbers correctly. And finally, the SVM confusion matrix shows that the model is a waste of time and not worth using when the multi-classification range is large.

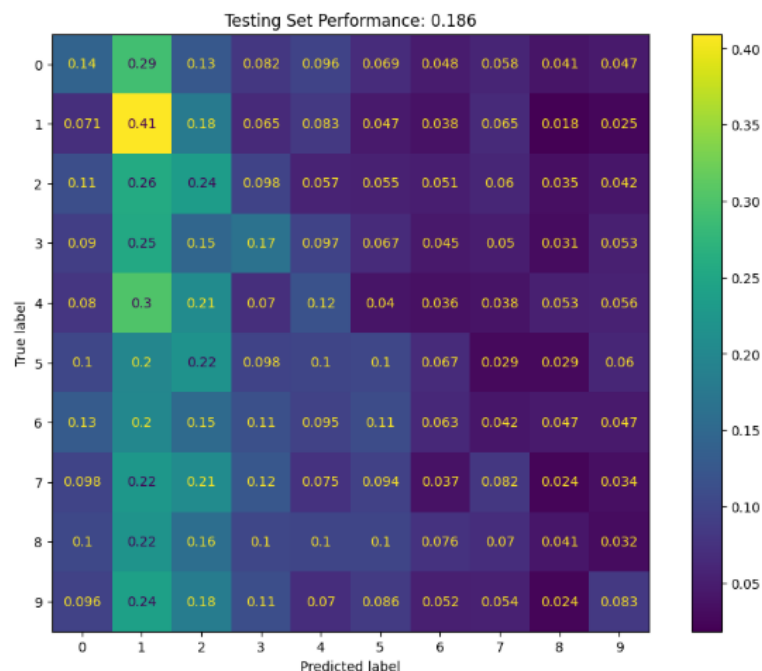


Figure 3.2: SVM Testing Confusion Matrix

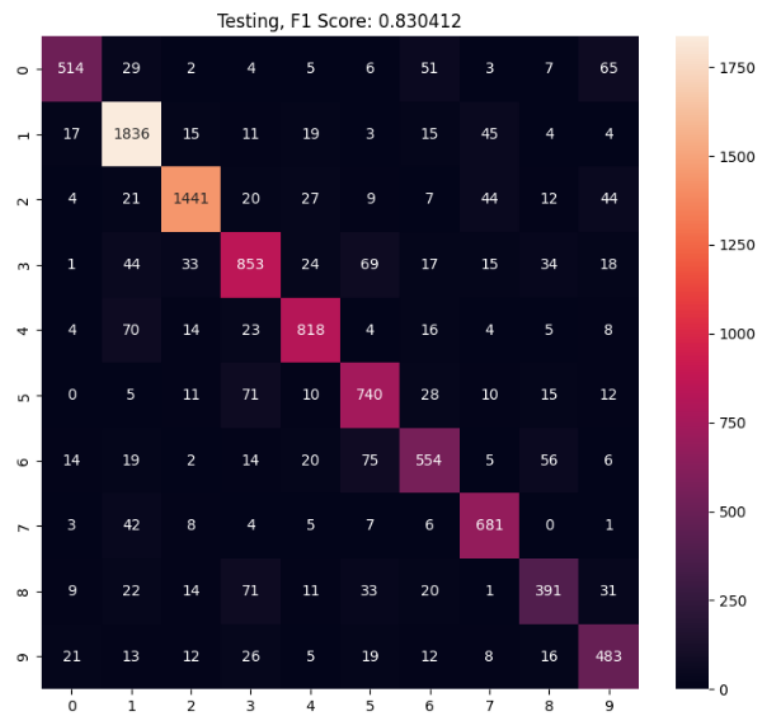


Figure 3.3: DCNN Testing Confusion Matrix

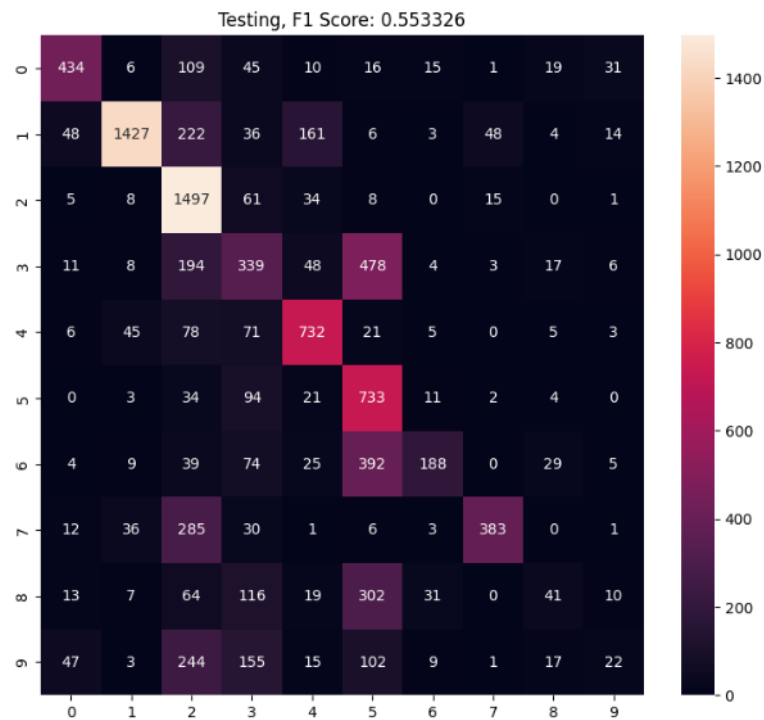


Figure 3.4: Augmented DCNN Testing Confusion Matrix

As seen in Figures 3.5 and 3.6, the unaugmented DCNN models' validation accuracy starts at 0.1 and rises about 0.2 every 10 epochs until about epochs 35, where it begins to settle to around 0.8 to 0.7 roughly. The augmented DCNN models' validation accuracy was quite the opposite where it flatlined around 0.2 accuracy for the first 40 epochs before learning and starting to increase to 0.6 around the 100 epochs mark. (See Appendix 3D & 3E for Train vs Validation Loss for DCNN models) (See appendix 4C for code to replicate similar results for Problem 3)

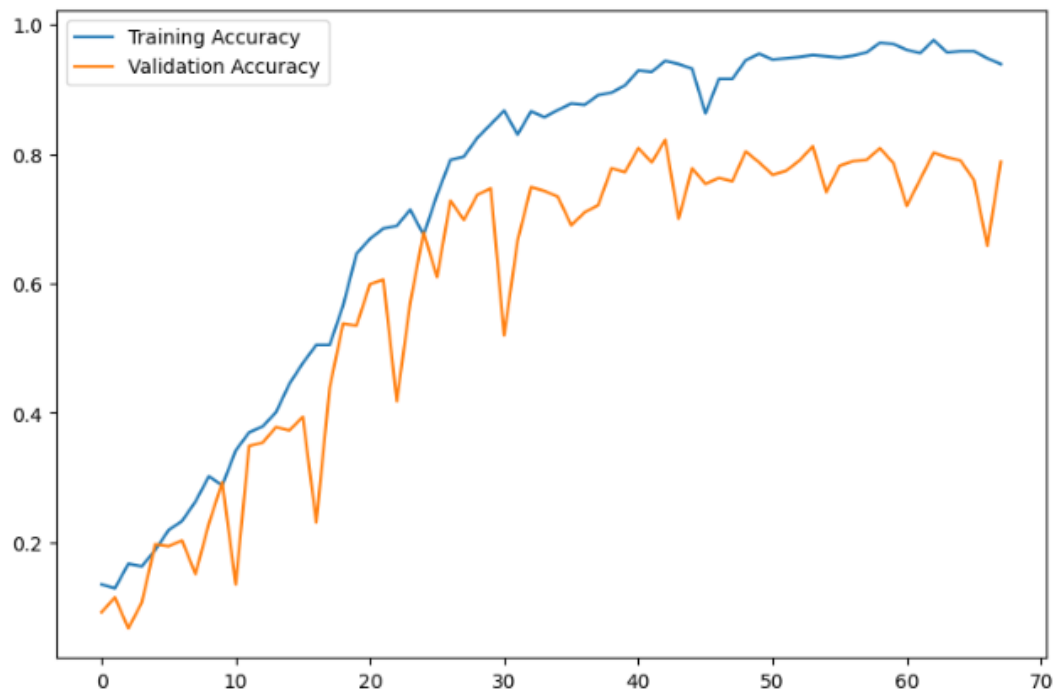


Figure 3.5: DCNN Training vs Validation Accuracy

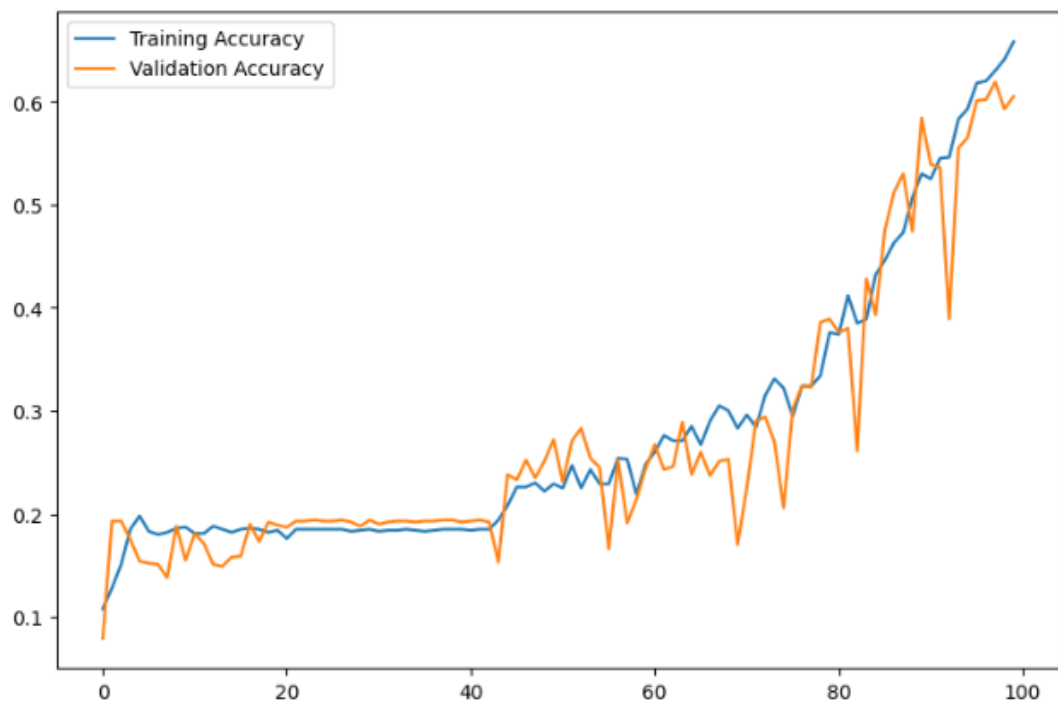


Figure 3.6: Augmented DCNN Training vs Validation Accuracy

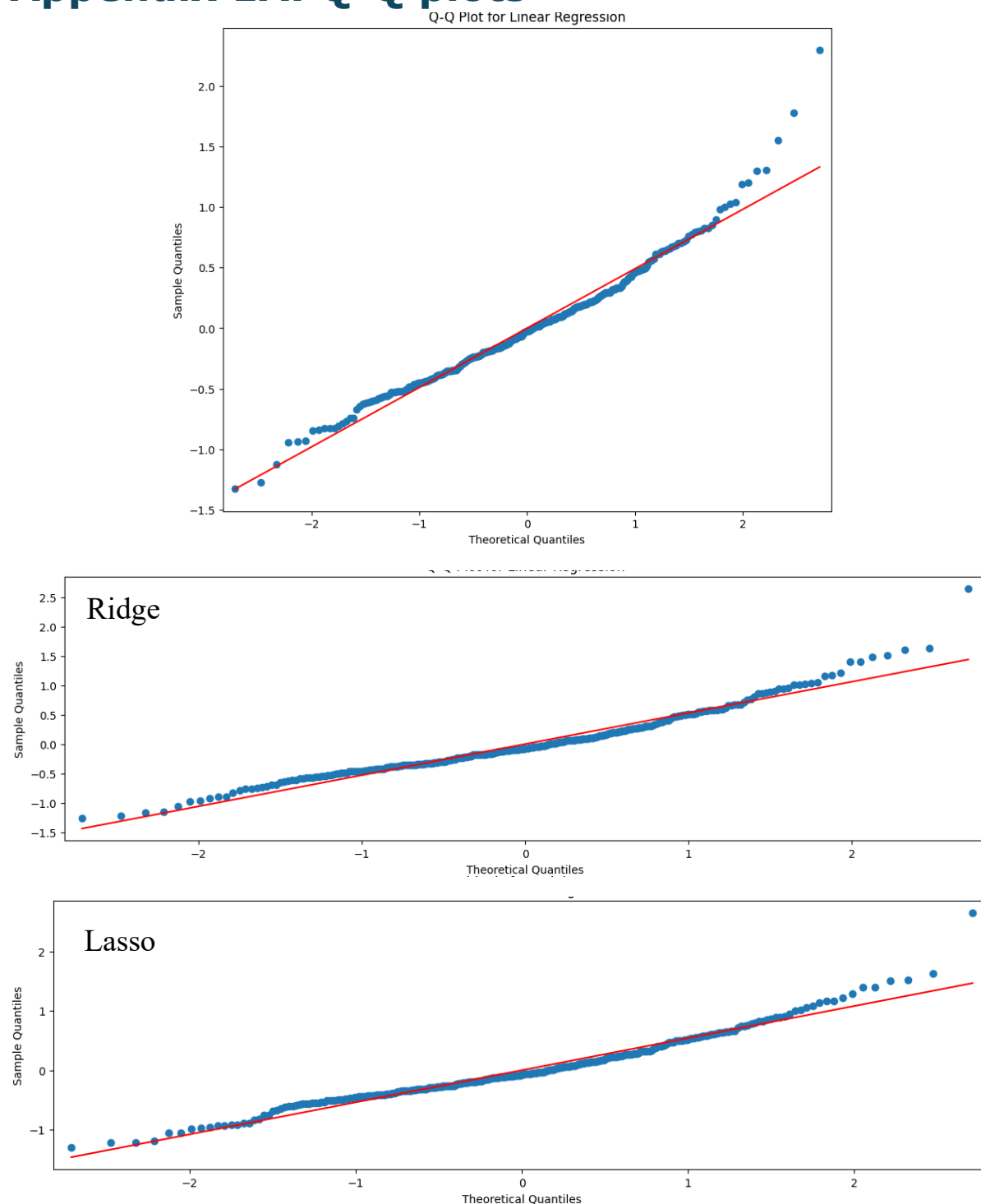
References:

- [1] S. Denman, “CAB420, Practical 2 - Practical Solution,” QUT, [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-2-practical-solutions?module_item_id=1798256. [Accessed 01 April 2025].
- [2] Z. Jaabi, “Data Standardization: How to Do It and Why It Matters,” builtin, 09 January 2025. [Online]. Available: <https://builtin.com/data-science/when-and-why-standardize-your-data#:~:text=Before%20Lasso%20and%20Ridge%20Regressions&text=Coefficients%20of%20variables%20with%20a,required%20before%20fitting%20both%20regressions..> [Accessed 01 April 2025].
- [3] S. Denman, “CAB420, Week 1 - Practical Solution,” QUT, [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-1-practical-solutions?module_item_id=1798249. [Accessed April 01 2025].
- [4] S. Olumide, “Root Mean Square Error (RMSE) In AI: What You Need To Know,” arize, 08 August 2023. [Online]. Available: <https://arize.com/blog-course/root-mean-square-error-rmse-what-you-need-to-know/#:~:text=Low%20RMSE%20values%20show%20that,mistakes%20and%20fewer%20accurate%20forecasts..> [Accessed 01 April 2025].
- [5] IGNEsa, “Crime Rates are Resilient,” IGNEsa, [Online]. Available: <https://ignesa.com/insights/crime-rate-and-ethical-crime-prediction/>. [Accessed 01 April 2025].
- [6] S. Denman, “CAB420, Practical 3 - Practical Solution,” QUT, [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-3-practical-solutions?module_item_id=1798263. [Accessed 02 April 2025].
- [7] Xbeat, “Mastering K-Nearest Neighbors Hyperparameters in Python,” Github, 12 September 2024. [Online]. Available: <https://github.com/xbeat/Machine-Learning/blob/main/Mastering%20K-Nearest%20Neighbors%20Hyperparameters%20in%20Python.md>. [Accessed 02 April 2025].

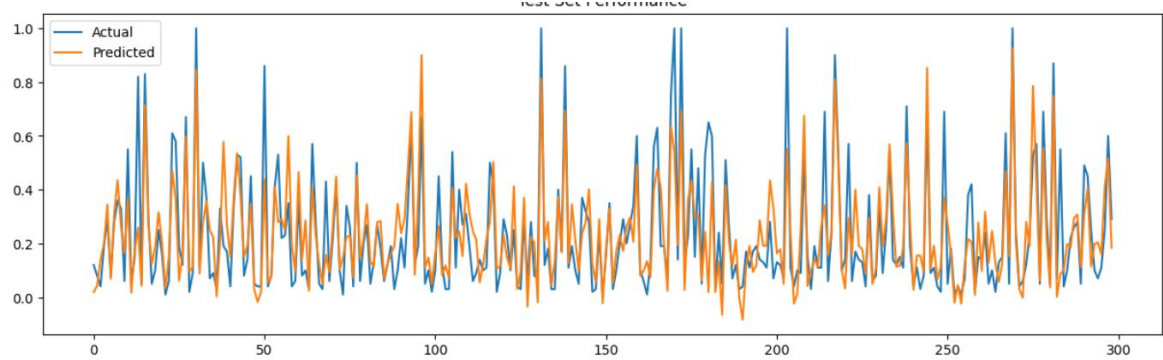
- [8] M. B. Fraj, “In Depth: Parameter tuning for Random Forest,” Medium, 22 December 2017. [Online]. Available: <https://medium.com/all-things-ai/in-depth-parameter-tuning-for-random-forest-d67bb7e920d>. [Accessed 02 April 2025].
- [9] P. Belagatti, “Understanding the Softmax Activation Function: A Comprehensive Guide,” SingleStore, 12 March 2024. [Online]. Available: <https://www.singlestore.com/blog/a-guide-to-softmax-activation-function/#:~:text=Softmax%20provides%20a%20well%2Ddefined,debugging%20of%20the%20neural%20network..> [Accessed 03 April 2025].
- [10] H. Yadav, “Dropout in Neural Networks,” towards data science, 05 July 2022. [Online]. Available: <https://towardsdatascience.com/dropout-in-neural-networks-47a162d621d9/>. [Accessed 05 April 2025].
- [11] R. Agarwal, “Complete Guide to the Adam Optimization Algorithm,” BuiltIn, 13 September 2023. [Online]. Available: <https://builtin.com/machine-learning/adam-optimization>. [Accessed 05 April 2025].
- [12] M. Sorokin, “Categorical Cross-Entropy: Unraveling its Potentials in Multi-Class Classification,” Medium, 17 January 2024. [Online]. Available: <https://medium.com/@vergotten/categorical-cross-entropy-unraveling-its-potentials-in-multi-class-classification-705129594a01#:~:text=Advantages%20of%20Categorical%20Cross%2DEntropy,model's%20confidence%20in%20its%20decisions..> [Accessed 05 April 2025].

APPENDICES:

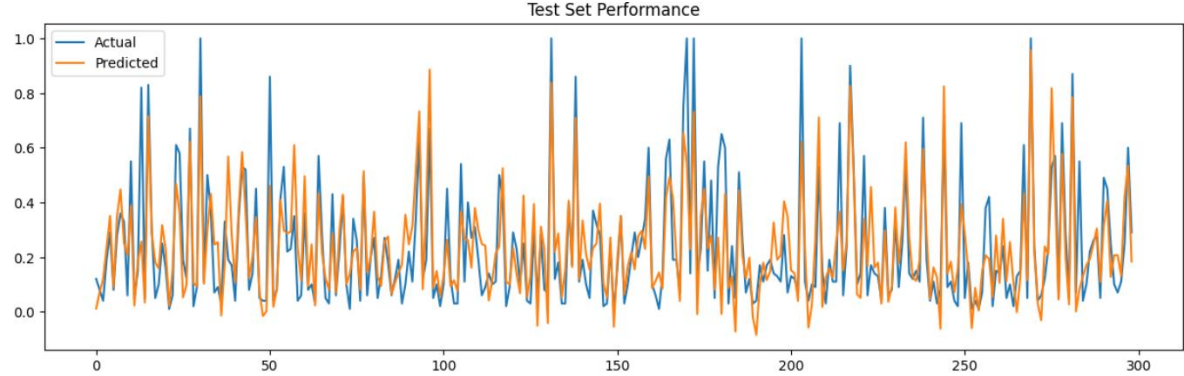
Appendix 1A: Q-Q plots



Appendix 1B: Ridge Test Set Performance



Appendix 1C: Lasso Test Set Performance



Appendix 1D: Linear Regression Summary

Model LR RMSE = 0.7297

Model LR R² Score = 0.7591

OLS Regression Results			
=====			
Dep. Variable:	ViolentCrimesPerPop	R-squared (uncentered):	0.759
Model:	OLS	Adj. R-squared (uncentered):	0.637
Method:	Least Squares	F-statistic:	6.239
Date:	Sat, 05 Apr 2025	Prob (F-statistic):	4.63e-28
Time:	09:35:57	Log-Likelihood:	-210.77
No. Observations:	298	AIC:	621.5
Df Residuals:	198	BIC:	991.2
Df Model:	100		
Covariance Type:	nonrobust		
=====			

Appendix 1E: Ridge Summary

RIDGE:
Best values on Validation Data set
Best Alpha =0.06135907273413163
Best R Squared = 0.7182056062081854
Best Adjusted = 0.5751627667199547
Best RMSE (val) = 0.14247714909139023
Best RMSE (test) = 0.1314902865929718
Best coefficients on the normalised model

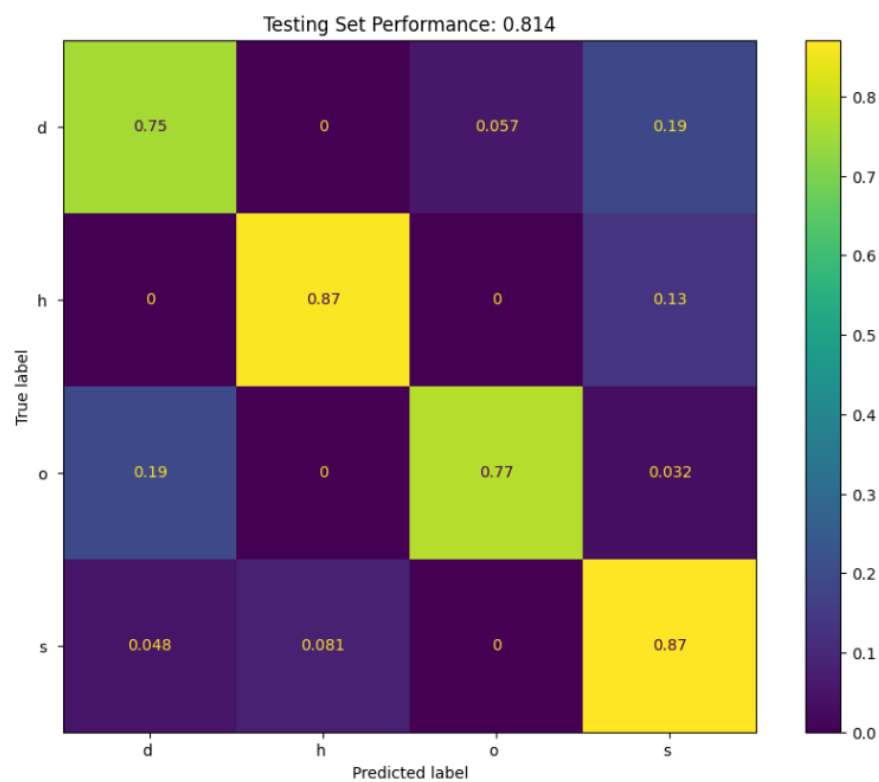
Appendix 1F: Lasso Summary

LASSO:
Best values on Validation Data set
Best Alpha =0.002364489412645407
Best R Squared = 0.7062344312663479
Best Adjusted = 0.5571148532289609
Best RMSE (val) = 0.14390977923618642
Best RMSE (test) = 0.1319091428887659
Best coefficients on the normalised model

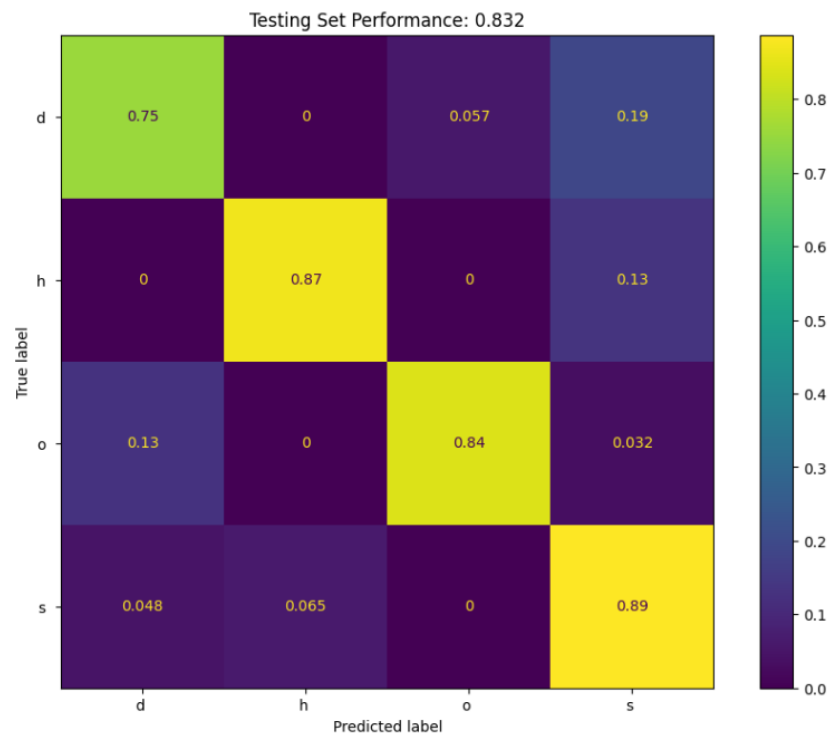
Appendix 2A: SVM Grid Confusion Matrix



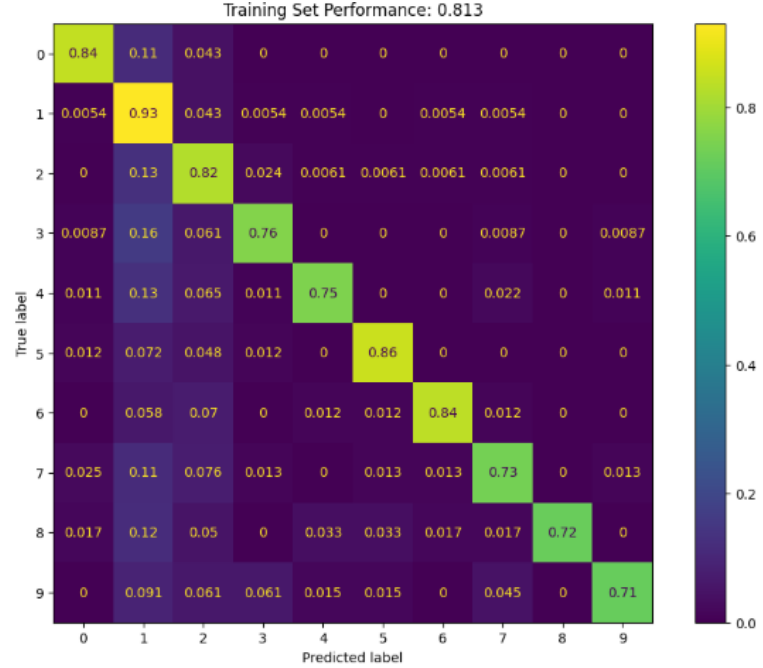
Appendix 2B: RF Grid Confusion Matrix



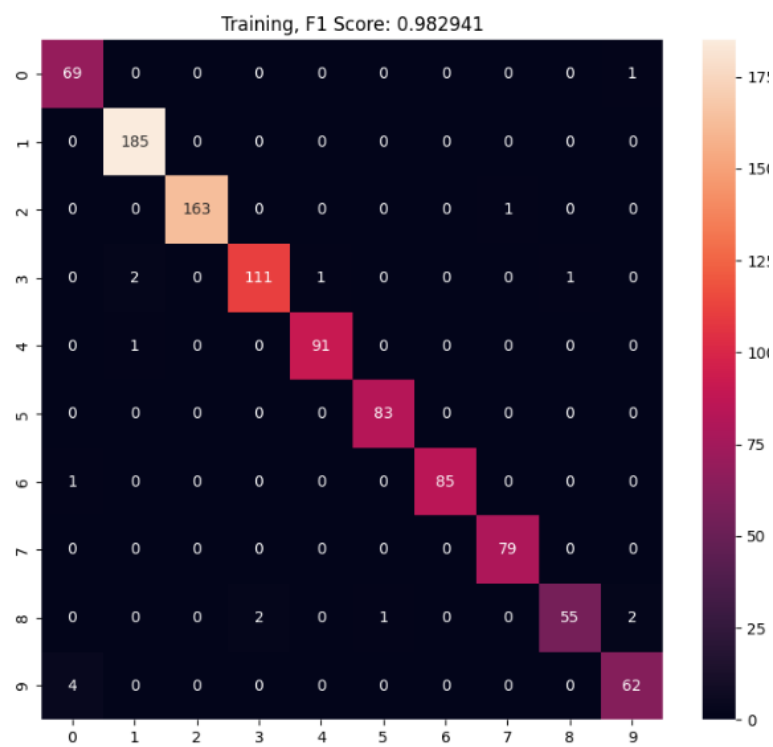
Appendix 2C: RF Trial Confusion Matrix



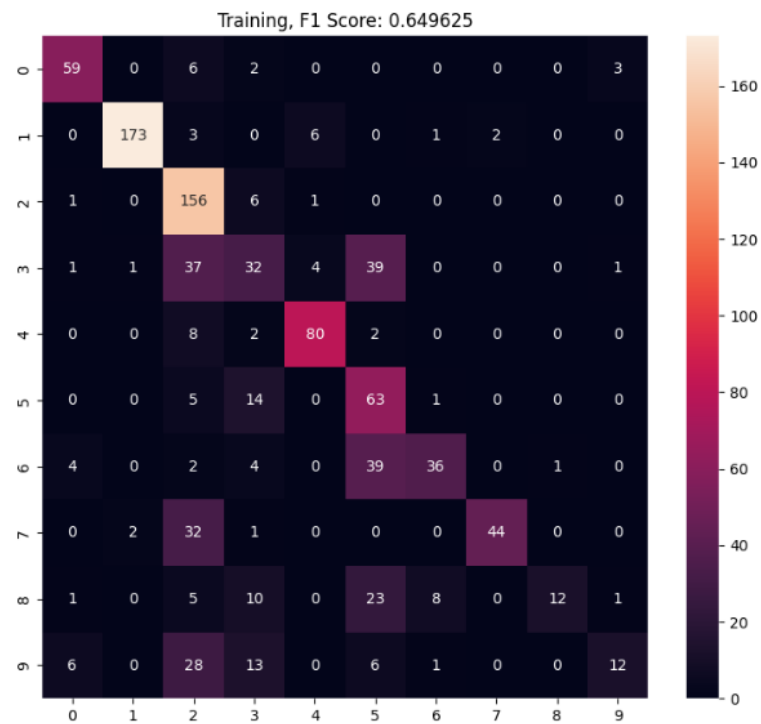
Appendix 3A: SVM training Confusion Matrix



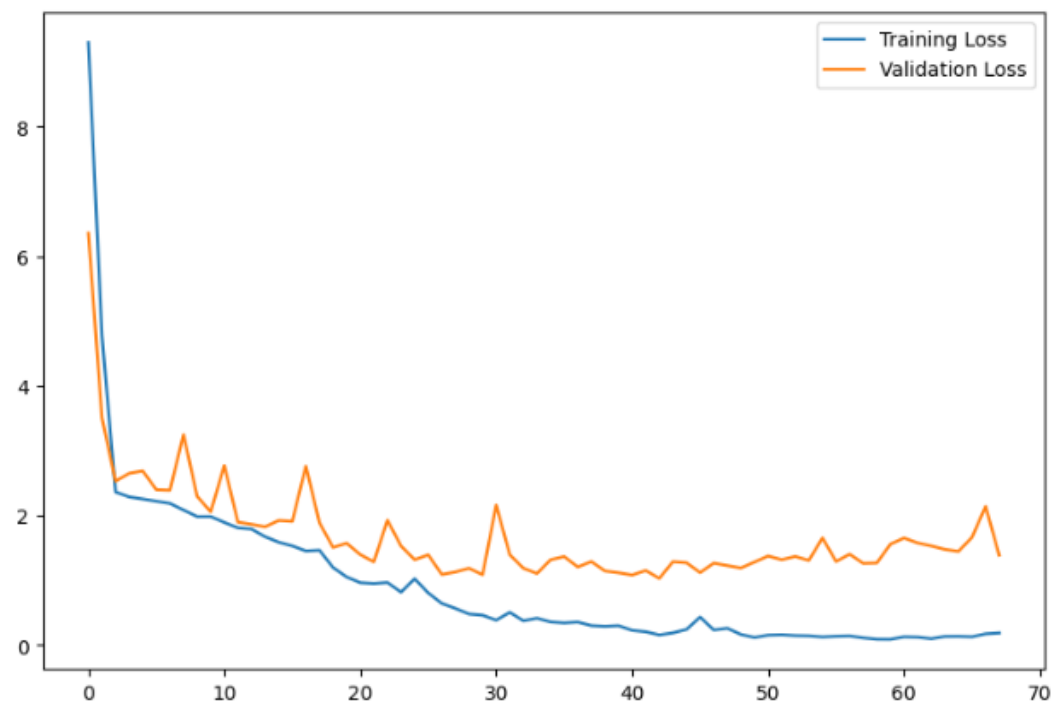
Appendix 3B: DCNN training Confusion Matrix



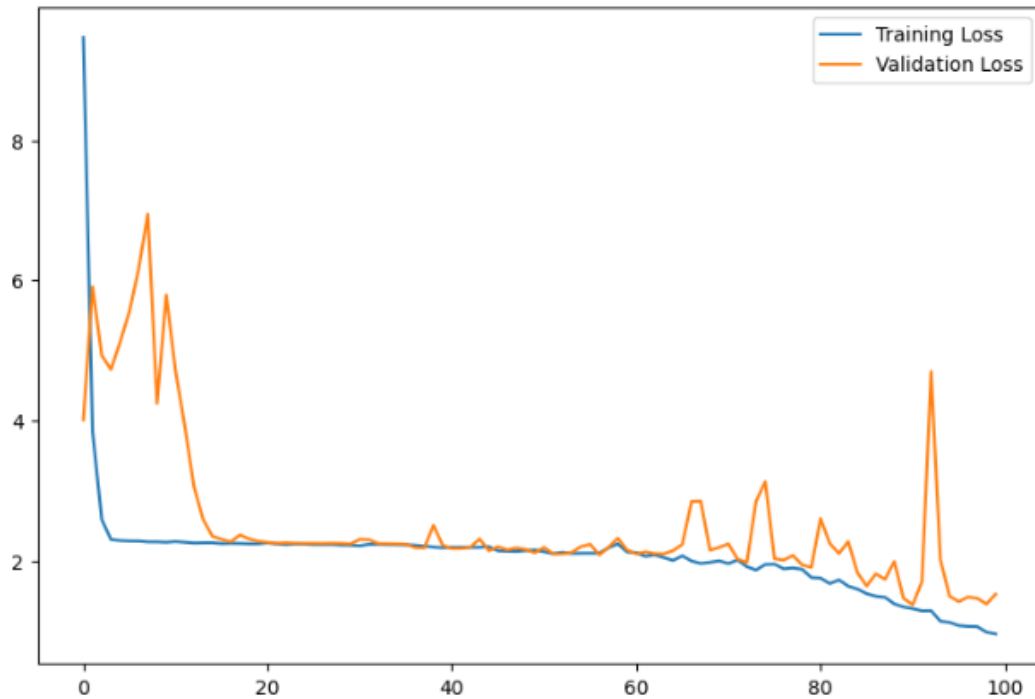
Appendix 3C: Augmented DCNN training Confusion Matrix



Appendix 3D: DCNN Training vs Validation Loss



Appendix 3E: Augmented DCNN Training vs Validation Loss



Appendix 4A: Problem 1 Code

```
from google.colab import drive
drive.mount('/content/drive')
import pandas
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns;
from statsmodels import api as sm
from scipy import stats
from sklearn.metrics import mean_squared_error, r2_score
import os
from datetime import datetime, date
import itertools

# load data
train =
pandas.read_csv('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q1
/communities_train.csv')
val =
pandas.read_csv('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q1
/communities_val.csv')
test =
pandas.read_csv('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q1
/communities_test.csv')

# Pull out X and y components of the data
# if you want to convert this data to numpy format, you can add .to_numpy()
to the end of each row below
X_train = train.iloc[:,0:-1]
Y_train = train.iloc[:,-1]
X_val = val.iloc[:,0:-1]
Y_val = val.iloc[:,-1]
X_test = test.iloc[:,0:-1]
Y_test = test.iloc[:,-1]

def standardise(data):
    mu = np.mean(data, axis=0)
    sigma = np.std(data, axis=0)
    scaled = (data - mu) / sigma
    return scaled, mu, sigma

X_train_std, mu_train_x, sigma_train_x = standardise(X_train)
Y_train_std, mu_train_y, sigma_train_y = standardise(Y_train)
X_val_std = (X_val - mu_train_x)/sigma_train_x
Y_val_std = (Y_val - mu_train_y)/sigma_train_y
X_test_std = (X_test - mu_train_x)/sigma_train_x
Y_test_std = (Y_test - mu_train_y)/sigma_train_y
```

```

LRMODEL = sm.OLS(Y_train_std, X_train_std)
model_LR_fit = LRMODEL.fit()
pred = model_LR_fit.predict(X_val_std)

# Measuring the accuracy of the model (i.e. RMSE or similar measures)
rmse = np.sqrt(mean_squared_error(Y_val_std, pred))
print(f'Model LR RMSE = {rmse:.4f}')
r2 = model_LR_fit.rsquared
print(f'Model LR R2 Score = {r2:.4f}')

# Looking at the properties of the fitted model, for example the R^2, and
if all terms are contributing to the model
print(model_LR_fit.summary())

# Q-Q Plot (Checking normality of residuals)
fig, ax = plt.subplots(figsize=(10,8))
sm.qqplot(model_LR_fit.resid, ax=ax, line='s')
plt.title('Q-Q Plot for Linear Regression')
plt.show()

# Residuals vs. Predictions Plot (Checking homoscedasticity)
fig, ax = plt.subplots(figsize=(10,8))
ax.scatter(model_LR_fit.predict(X_train_std), model_LR_fit.resid)
ax.set_xlabel('Predictions')
ax.set_ylabel('Residuals')
ax.axhline(y=0, color='r', linestyle='--')
plt.title('Residuals vs Predictions')
plt.show()

# Plot
plt.figure(figsize=(6, 5))
plt.bar(['Train', 'Validation', 'Test'], [train_rmse_linear,
val_rmse_linear, test_rmse_linear], color='skyblue')
plt.title('Linear Regression RMSE')
plt.ylabel('RMSE')
plt.grid(axis='y')
plt.ylim(0, max(train_rmse_linear, val_rmse_linear, test_rmse_linear) +
0.1)
plt.show()

def rmse(actual, pred):
    return np.sqrt(mean_squared_error(actual, pred))

def r_squared(actual, predicted):
    r2 = r2_score(actual, predicted)
    return r2

def adj_r2(actual, predicted, n, p):
    r2 = r2_score(actual, predicted)
    adjr2 = 1 - (1 - r2) * (n - 1) / (n - p - 1);
    return adjr2

```

```

def evaluate_regularisation(x_train, y_train, x_val, y_val, x_test, y_test,
                           response_mu, response_sigma, alpha_list,
                           L1_L2):

    # Ridge: L1_L2 = 0
    # Lasso: L1_L2 = 1
    model = sm.OLS(y_train, x_train)
    best_rmse = 10e12
    best_alpha = []
    best_coefs = []
    rmse_val = []
    rmse_train = []
    coefs = []
    for alpha in alpha_list:
        model_cross_fit = model.fit_regularized(alpha=alpha, L1_wt=L1_L2)
        train_pred = model_cross_fit.predict(x_train)
        val_pred = model_cross_fit.predict(x_val)
        rmse_train.append(np.sqrt(mean_squared_error(y_train, train_pred)))
        rmse_val.append(np.sqrt(mean_squared_error(y_val, val_pred)))
        coefs.append(model_cross_fit.params)
        if rmse_val[-1] < best_rmse:
            best_rmse = rmse_val[-1]
            best_alpha = alpha
            best_coefs = model_cross_fit.params

    print('Best values on Validation Data set')
    slope = np.array(best_coefs[0:]).reshape(-1, 1)
    pred_val_rescaled = (x_val @ slope) * response_sigma + response_mu
    pred_test_rescaled = (x_test @ slope) * response_sigma + response_mu
    pred_train_rescaled = (x_train @ slope) * response_sigma + response_mu
    best_r2 = r_squared(y_train * response_sigma + response_mu,
                       pred_train_rescaled)
    best_adj_r2 = adj_r2(y_train * response_sigma + response_mu,
                       pred_train_rescaled, x_train.shape[0], x_train.shape[1])
    best_val_rmse = np.sqrt(mean_squared_error(y_val * response_sigma +
                                               response_mu, pred_val_rescaled))
    best_test_rmse = np.sqrt(mean_squared_error(y_test * response_sigma +
                                                response_mu, pred_test_rescaled))
    print('Best Alpha = {}'.format(best_alpha))
    print('Best R Squared = {}'.format(best_r2))
    print('Best Adjusted = {}'.format(best_adj_r2))
    print('Best RMSE (val) = {}'.format(best_val_rmse))
    print('Best RMSE (test) = {}'.format(best_test_rmse))
    print('Best coefficients on the normalised model')
    print('Best slope = {}'.format(slope))

    fig, axs = plt.subplots(5, figsize=(15, 25))
    axs[0].plot(np.array(alpha_list), rmse_train)
    axs[0].plot(np.array(alpha_list), rmse_val)
    axs[0].legend(['Training', 'Validation'])
    axs[0].set_title('RMSE vs Lambda')
    axs[0].set_xlabel('Lambda')
    axs[0].set_ylabel('RMSE')

    # plot prediction and true values for test set
    axs[1].plot((Y_test_std*response_sigma + response_mu))
    axs[1].plot((X_test_std @ slope) * response_sigma + response_mu)
    axs[1].legend(['Actual', 'Predicted'])
    axs[1].set_title('Test Set Performance')

    # plotting the Q-Q plot

```

```
train_pred = (X_train_std @ slope).values.reshape(Y_train.shape)
resid = Y_train_std - train_pred
sm.qqplot(resid, ax=axis[2], line='s')
axis[2].set_title('Q-Q Plot for Linear Regression')

# plot the residuals as well
axis[3].scatter(train_pred, resid)
axis[3].set_title('Residuals for training set')
axis[3].set_xlabel('Predicted')
axis[3].set_ylabel('Residuals')

# trace plot of coefficients
axis[4].plot(np.array(alpha_list), coeffs)
axis[4].set_title('Trace Plot of Coefficients')
axis[4].set_xlabel('Lambda')
axis[4].set_ylabel('Coefficient Value')

alphas = np.logspace(-10, 0, 100)

## RIDGE
print("RIDGE:")
evaluate_regularisation(X_train_std, Y_train_std, X_val_std, Y_val_std,
X_test_std, Y_test_std, mu_train_y, sigma_train_y, alphas, 0)

## LASSO
print("LASSO:")
evaluate_regularisation(X_train_std, Y_train_std, X_val_std, Y_val_std,
X_test_std, Y_test_std, mu_train_y, sigma_train_y, alphas, 1)
```

Appendix 4B: Problem 2 Code

```
from google.colab import drive
drive.mount('/content/drive')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
import string
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.svm import SVC
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score, recall_score, f1_score,
classification_report
from sklearn.model_selection import GridSearchCV
from scipy.stats import norm
from sklearn.model_selection import PredefinedSplit
from sklearn.metrics import classification_report

# load data
train =
pd.read_csv('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q2/training.csv')
val =
pd.read_csv('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q2/validation.csv')
test =
pd.read_csv('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q2/testing.csv')

# pull out X and y data, convert to numpy
X_train = train.iloc[:,1:].to_numpy()
Y_train = train.iloc[:,0].to_numpy()
X_val = val.iloc[:,1:].to_numpy()
Y_val = val.iloc[:,0].to_numpy()
X_test = test.iloc[:,1:].to_numpy()
Y_test = test.iloc[:,0].to_numpy()
```

```

# Q2 asks you to use a grid search for each model. You are to evaluate the
models on the validation set, and train on the train
# set. If you wish to use sklearn's GridSearchCV, you will need to setup a
PredefinedSplit to pass into this. An appropriate
# PredefinedSplit object is created below.
X_train_and_val = np.vstack((X_train, X_val))
Y_train_and_val = np.hstack((Y_train, Y_val))
mask = np.zeros(len(Y_train_and_val))
mask[0:len(Y_train)] = -1
train_val_split = PredefinedSplit(mask)
def eval_model(model, X_train, Y_train, X_test, Y_test):
    fig = plt.figure(figsize=[25, 8])
    ax = fig.add_subplot(1, 2, 1)
    conf = ConfusionMatrixDisplay.from_estimator(model, X_train, Y_train,
normalize='true', ax=ax)
    conf.ax_.set_title('Training Set Performance: %1.3f' %
(sum(model.predict(X_train) == Y_train)/len(Y_train)));
    ax = fig.add_subplot(1, 2, 2)
    conf = ConfusionMatrixDisplay.from_estimator(model, X_test, Y_test,
normalize='true', ax=ax)
    conf.ax_.set_title('Testing Set Performance: %1.3f' %
(sum(model.predict(X_test) == Y_test)/len(Y_test)));
    print(classification_report(Y_test, model.predict(X_test)))

#for k in [1,2,5,10]:
    #cknn = KNeighborsClassifier(n_neighbors=k, weights='uniform')
    #cknn.fit(X_train, Y_train)
    #eval_model(cknn, X_train, Y_train, X_test, Y_test)
#for k in [1,2,5,10]:
    #cknn = KNeighborsClassifier(n_neighbors=k, weights='distance')
    #cknn.fit(X_train, Y_train)
    #eval_model(cknn, X_train, Y_train, X_test, Y_test)

## WEIGHT(DISTANCE) = BEST
#or k in [1,2,5,10]:
    #cknn = KNeighborsClassifier(n_neighbors=k, weights='distance',
metric='minkowski')
    #cknn.fit(X_train, Y_train)
    #eval_model(cknn, X_train, Y_train, X_test, Y_test)
#for k in [1,2,5,10]:
    #cknn = KNeighborsClassifier(n_neighbors=k, weights='distance',
metric='manhattan')
    #cknn.fit(X_train, Y_train)
    #eval_model(cknn, X_train, Y_train, X_test, Y_test)

## Check K = 1,2,4,8, weights = distance, metric = both
#for k in [1,2,4]:
    #cknn = KNeighborsClassifier(n_neighbors=k, weights='distance',
metric='minkowski')
    #cknn.fit(X_train, Y_train)
    #eval_model(cknn, X_train, Y_train, X_test, Y_test)
#for k in [1,2,4]:
    #cknn = KNeighborsClassifier(n_neighbors=k, weights='distance',
metric='manhattan')
    #cknn.fit(X_train, Y_train)
    #eval_model(cknn, X_train, Y_train, X_test, Y_test)

```

```
cknnBEST = KNeighborsClassifier(n_neighbors=4, weights='distance',
metric='minkowski')
cknnBEST.fit(X_train, Y_train)
print("BEST: k = 4, weight = distance, metric = minkowski")
eval_model(cknnBEST, X_train, Y_train, X_test, Y_test)

param_grid = [
    {'n_neighbors': list(range(3,198)),
     'weights': ['uniform', 'distance'],
     'metric': ['minkowski', 'manhattan']}
]
cknn = KNeighborsClassifier()
grid_search = GridSearchCV(cknn, param_grid, cv=train_val_split) # Specify
scoring metric
grid_search.fit(X_train_and_val, Y_train_and_val)
grid_search.cv_results_
best_index = np.argmin(grid_search.cv_results_['rank_test_score'])
params = grid_search.cv_results_['params'][best_index]
print("Best Parameters:", params)
cknn = KNeighborsClassifier().set_params(**params)
cknn.fit(X_train, Y_train)
eval_model(cknn, X_train, Y_train, X_test, Y_test)

param_grid = [
    {'C': [0.01,0.1, 1, 10, 100,1000], 'kernel': ['linear']},
    {'C': [0.01,0.1, 1, 10, 100,1000], 'gamma': [1,0.1, 0.01, 0.001],
'kernel': ['rbf']},
    {'C': [0.01,0.1, 1, 10, 100,1000], 'degree': [1,3, 4,
5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39], 'kernel': ['poly']},
]
svm = SVC(class_weight='balanced')
grid_search = GridSearchCV(svm, param_grid, cv=train_val_split)
grid_search.fit(X_train_and_val, Y_train_and_val)
grid_search.cv_results_
best_system = np.argmin(grid_search.cv_results_['rank_test_score'])
params = grid_search.cv_results_['params'][best_system]
print(params)
svm = SVC().set_params(**params)
svm.fit(X_train, Y_train)
eval_model(svm, X_train, Y_train, X_test, Y_test)
```

```

## ESTIMATOR = 1, DEPTH = 1, 2, 5, 10
#for d in [1,2,5,10]:
    #rf = RandomForestClassifier(n_estimators=1, max_depth=d,
    random_state=0).fit(X_train, Y_train)
    #rf.fit(X_train, Y_train)
    #eval_model(rf, X_train, Y_train, X_test, Y_test)
#for d in [1,2,5,10]:
    #rf = RandomForestClassifier(n_estimators=10, max_depth=d,
    random_state=0).fit(X_train, Y_train)
    #rf.fit(X_train, Y_train)
    #eval_model(rf, X_train, Y_train, X_test, Y_test)
#for d in [1,2,5,10]:
    #rf = RandomForestClassifier(n_estimators=100, max_depth=d,
    random_state=0).fit(X_train, Y_train)
    #rf.fit(X_train, Y_train)
    #eval_model(rf, X_train, Y_train, X_test, Y_test)
#for d in [1,2,5,10]:
    #rf = RandomForestClassifier(n_estimators=1000, max_depth=d,
    random_state=0).fit(X_train, Y_train)
    #rf.fit(X_train, Y_train)
    #eval_model(rf, X_train, Y_train, X_test, Y_test)

## Best depth = 5
#for e in [30,40,50,60,70,80]:
    #rf = RandomForestClassifier(n_estimators=e, max_depth=5,
    random_state=0).fit(X_train, Y_train)
    #rf.fit(X_train, Y_train)
    #eval_model(rf, X_train, Y_train, X_test, Y_test)

rfBEST = RandomForestClassifier(n_estimators=40, max_depth=5,
    random_state=0).fit(X_train, Y_train)
print("BEST: estimator = 30, depth = 5")
eval_model(rfBEST, X_train, Y_train, X_test, Y_test)

rfc = RandomForestClassifier(random_state=0)
param_grid = {
    'n_estimators': list(range(1,100)),
    'max_depth' : list(range(10,20))
}
grid_search = GridSearchCV(rfc, param_grid,cv=train_val_split)
grid_search.fit(X_train_and_val, Y_train_and_val)
grid_search.cv_results_
best_system = np.argmin(grid_search.cv_results_['rank_test_score'])
params = grid_search.cv_results_['params'][best_system]
print(params)
rfc = RandomForestClassifier().set_params(**params)
rfc.fit(X_train, Y_train)
eval_model(rfc, X_train, Y_train, X_test, Y_test)

```

Appendix 4C: Problem 3 Code

```
#
# Utility functions for CAB420, Assignment 1A, Q3
# Author: Simon Denman (s.denman@qut.edu.au)
#
from google.colab import drive
drive.mount('/content/drive')
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

from scipy.io import loadmat          # to load mat files
import matplotlib.pyplot as plt       # for plotting
import numpy as np                    # for reshaping, array manipulation
import cv2                            # for colour conversion
import tensorflow as tf               # for bulk image resize
import keras
import seaborn as sns
from tensorflow import keras
from tensorflow.keras import layers, models, callbacks
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import precision_score, recall_score, f1_score,
classification_report
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import seaborn as sns

# Load data for Q3
# train_path: path to training data mat file
# test_path: path to testing data mat file
#
# returns: arrays for training and testing X and Y data
#
def load_data(train_path, val_path, test_path):

    # load files
    train = loadmat(train_path)
    val = loadmat(val_path)
    test = loadmat(test_path)

    # pull out training set and scale X data
    train_X = train['train_X'] / 255.0
    train_Y = train['train_y'].reshape(-1)

    # pull out validation set and scale X data
    val_X = val['val_X'] / 255.0
    val_Y = val['val_y'].reshape(-1)

    # pull out test set and scale X data
    test_X = test['test_X'] / 255.0
    test_Y = test['test_y'].reshape(-1)

    # return loaded data
    return train_X, train_Y, val_X, val_Y, test_X, test_Y
```

```
# vectorise an array of images, such that the shape is changed from {samples,
width, height, channels} to
# (samples, width * height * channels)
# images: array of images to vectorise
#
# returns: vectorised array of images
#
def vectorise(images):
    # use numpy's reshape to vectorise the data
    return np.reshape(images, [len(images), -1])

# Plot some images and their labels. Will plot the first 100 samples in a
10x10 grid
# x: array of images, of shape (samples, width, height, channels)
# y: labels of the images
#
def plot_images(x, y):
    fig = plt.figure(figsize=[15, 18])
    for i in range(100):
        ax = fig.add_subplot(10, 10, i + 1)
        ax.imshow(x[i,:])
        ax.set_title(y[i])
        ax.axis('off')

# Resize an array of images
# images: array of images, of shape (samples, width, height, channels)
# new_size: tuple of the new size, (new_width, new_height)
#
# returns: resized array of images, (samples, new_width, new_height,
channels)
#
def resize(images, new_size):
    # tensorflow has an image resize funtion that can do this in bulk
    # note the conversion back to numpy after the resize
    return tf.image.resize(images, new_size).numpy()

# Convert images to grayscale
# images: array of colour images to convert, of size (samples, width,
height, 3)
#
# returns: array of converted images, of size (samples, width, height, 1)
#
def convert_to_grayscale(images):
    # storage for converted images
    gray = []
    images = images.numpy()
    # loop through images
    for i in range(len(images)):
        # convert each image using openCV
        gray.append(cv2.cvtColor(images[i,:], cv2.COLOR_BGR2GRAY))
    # pack converted list as an array and return
    return np.expand_dims(np.array(gray), axis = -1)
```

```
from sklearn.svm import SVC
from time import process_time

# load data
train_X,      train_Y, val_X,      val_Y,      test_X,      test_Y      =
load_data('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q3/q3_train.mat',
'/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q3/q3_validation.mat',
'/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1A/Q3/q3_test.mat')
train_YY = to_categorical(train_Y, num_classes=10)
val_YY = to_categorical(val_Y, num_classes=10)
test_YY = to_categorical(test_Y, num_classes=10)
# any resize, colour change, etc, would go here
# if you do anything, do the same thing to the training, validation, and
test sets

# vectorise data
# If you do any resize, reshape, etc of the data prior to putting this into
your DCNN, change this code to
# vectorise that version of the data. The same data should be used by all
models for a fair comparison; though
# you will only vectorise the data for the SVM (i.e. the DCNN will get the
data as images).
train_vector_X = vectorise(train_X)
val_vector_X = vectorise(val_X)
test_vector_X = vectorise(test_X)
def eval_SVM_model(model, X_train, Y_train, X_test, Y_test):
    fig = plt.figure(figsize=[25, 8])
    ax = fig.add_subplot(1, 2, 1)
    conf = ConfusionMatrixDisplay.from_estimator(model, X_train, Y_train,
normalize='true', ax=ax)
    conf.ax_.set_title('Training Set Performance: %1.3f' %
(sum(model.predict(X_train) == Y_train)/len(Y_train)));
    ax = fig.add_subplot(1, 2, 2)
    conf = ConfusionMatrixDisplay.from_estimator(model, X_test, Y_test,
normalize='true', ax=ax)
    conf.ax_.set_title('Testing Set Performance: %1.3f' %
(sum(model.predict(X_test) == Y_test)/len(Y_test)));
    print(classification_report(Y_test, model.predict(X_test)))

# train the SVM
# Note that you are not required to tune the SVM, just use it as it is. This
also means that we do not need the validation
# set for the SVM as we're not selecting hyper-parameters
# measure the start time of training
svm_train_start = process_time()
# run the training - note that you don't need to change any parameters here
except potentially for the input data is you
# resize, change colour, etc
svm = SVC(C = 1, kernel = 'linear').fit(train_vector_X, train_Y)
# record the end time of the training
svm_train_end = process_time()
# predict on the training set
train_predictions = svm.predict(train_vector_X)
# measure of the time of the training set prediction
svm_train_pred_end = process_time()
```

```
# predict on the test set
test_predictions = svm.predict(test_vector_X)
# measure the end time on the prediction of the test set
svm_test_pred_end = process_time()

# check how long the SVM took to train
svm_train_time = svm_train_end - svm_train_start
# check how long the SVM took to perform inference on the training set
svm_inference_train_time = svm_train_pred_end - svm_train_end
# check how long the SVM took to perform inference on the testing set
svm_inference_test_time = svm_test_pred_end - svm_train_pred_end
# print some times
eval_SVM_model(svm, train_vector_X, train_Y, test_vector_X, test_Y)
print('Training Time: %f\nInference Time (training set): %f\nInference Time
(testing set): %f' % \
      (svm_train_time, svm_inference_train_time, svm_inference_test_time))

train_X_inverted = 1.0 - train_X
print("inverted",train_X_inverted.shape)
plot_images(train_X_inverted, train_Y)

rotate_layer_10 = tf.keras.layers.RandomRotation(factor=10/360.0)
train_X_rotated10 = rotate_layer_10(train_X)
print("rotated",train_X_rotated10.shape)
plot_images(train_X_rotated10, train_Y)

rotate_layer_20 = tf.keras.layers.RandomRotation(factor=20/360.0)
train_X_rotated20 = rotate_layer_20(train_X)
print("rotated",train_X_rotated20.shape)
plot_images(train_X_rotated20, train_Y)

rotate_layer_30 = tf.keras.layers.RandomRotation(factor=30/360.0)
train_X_rotated30 = rotate_layer_30(train_X)
print("rotated",train_X_rotated30.shape)
plot_images(train_X_rotated30, train_Y)

train_X_rotated30_inverted = 1.0 - train_X_rotated30
print("rotated gray scale",train_X_rotated30_inverted.shape)
plot_images(train_X_rotated30_inverted, train_Y)

## EVALUATION FUNCTION
def eval_model(model, train, train_y, test, test_y):
    fig = plt.figure(figsize=[20, 8])
    ax = fig.add_subplot(1, 2, 1)
    pred = model.predict(train, verbose=False);
    indexes = tf.argmax(pred, axis=1)
    gt_idx = tf.argmax(train_y, axis=1)
    confusion_mtx = tf.math.confusion_matrix(gt_idx, indexes)
    sns.heatmap(confusion_mtx, xticklabels=range(10), yticklabels=range(10),
annot=True, fmt='g', ax=ax)
    ax.set_title('Training, F1 Score: %f' % f1_score(gt_idx, indexes,
average='weighted'))
    ax = fig.add_subplot(1, 2, 2)
    pred = model.predict(test, verbose=False);
    indexes = tf.argmax(pred, axis=1)
    gt_idx = tf.argmax(test_y, axis=1)
    confusion_mtx = tf.math.confusion_matrix(gt_idx, indexes)
```

```

sns.heatmap(confusion_mtx, xticklabels=range(10), yticklabels=range(10),
annot=True, fmt='g', ax=ax)
ax.set_title('Testing, F1 Score: %f' % f1_score(gt_idx, indexes,
average='weighted'))

## NO AUGMENTATION
# Model parameters
from time import process_time
inputs = keras.Input(shape=(32, 32, 3), name='img')
x = layers.Conv2D(128, (3, 3), padding='same', activation='relu')(inputs)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(128, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Conv2D(256, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(256, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Conv2D(512, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(512, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Flatten()(x)
x = layers.Dense(512, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(10, activation='softmax')(x)

# Build model
model_vgg = keras.Model(inputs=inputs, outputs=outputs, name='digit_vgg')
early_stop = callbacks.EarlyStopping(monitor='val_loss', patience=25,
restore_best_weights=True)
model_vgg.compile(loss='categorical_crossentropy',
optimizer=keras.optimizers.Adam(), metrics=['accuracy'])
DCNN_train_start = process_time()
history = model_vgg.fit(train_X, train_YY, batch_size=32, epochs=100,
validation_data=(val_X, val_YY), callbacks=[early_stop])
DCNN_train_end = process_time()
DCNN_train_time = DCNN_train_end - DCNN_train_start

# PLOTS
fig = plt.figure(figsize=[20, 6])
ax = fig.add_subplot(1, 2, 1)
ax.plot(history.history['loss'], label="Training Loss")
ax.plot(history.history['val_loss'], label="Validation Loss")
ax.legend()
ax = fig.add_subplot(1, 2, 2)
ax.plot(history.history['accuracy'], label="Training Accuracy")
ax.plot(history.history['val_accuracy'], label="Validation Accuracy")
ax.legend()
DCNN_test_start = process_time()
eval_model(model_vgg, train_X, train_YY, test_X, test_YY)
DCNN_test_end = process_time()
DCNN_test_time = DCNN_test_end - DCNN_test_start
print('Training Time: %f' % (DCNN_train_time))
print('Test Time: %f' % (DCNN_test_time))

```

```

## WITH AUGMENTATION
# Model parameters
from time import process_time
inputs = keras.Input(shape=(32, 32, 3), name='img')
x = layers.Conv2D(128, (3, 3), padding='same', activation='relu')(inputs)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(128, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Conv2D(256, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(256, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Conv2D(512, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(512, (3, 3), padding='same', activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Flatten()(x)
x = layers.Dense(512, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(10, activation='softmax')(x)

# Build model
model_vgg2 = keras.Model(inputs=inputs, outputs=outputs, name='digit_vgg')
early_stop = callbacks.EarlyStopping(monitor='val_loss', patience=25,
restore_best_weights=True)
model_vgg2.compile(loss='categorical_crossentropy',
optimizer=keras.optimizers.Adam(), metrics=['accuracy'])
DCNNA_train_start = process_time()
history = model_vgg2.fit(train_X_rotated30_inverted, train_YY,
batch_size=32, epochs=100, validation_data=(val_X,
val_YY), callbacks=[early_stop])
DCNNA_train_end = process_time()
DCNNA_train_time = DCNNA_train_end - DCNNA_train_start

# PLOTS
fig = plt.figure(figsize=[20, 6])
ax = fig.add_subplot(1, 2, 1)
ax.plot(history.history['loss'], label="Training Loss")
ax.plot(history.history['val_loss'], label="Validation Loss")
ax.legend()
ax = fig.add_subplot(1, 2, 2)
ax.plot(history.history['accuracy'], label="Training Accuracy")
ax.plot(history.history['val_accuracy'], label="Validation Accuracy")
ax.legend()
DCNNA_test_start = process_time()
eval_model(model_vgg2, train_X_rotated30_inverted, train_YY, test_X,
test_YY)
DCNNA_test_end = process_time()
DCNNA_test_time = DCNNA_test_end - DCNNA_test_start
print('Training Time: %f' % (DCNNA_train_time))
print('Test Time: %f' % (DCNNA_test_time))

```